Launch Pad Levels V2

Multi-Timeframe Order Flow ACSIL Study

Functional Specification Document

Version:	V2.0
Study Name:	K_LaunchPadLevels_V2
Document Date:	November 01, 2025
Schema Version:	4
Platform:	Sierra Chart ACSIL
Classification:	Order Flow Analysis / Price Level Detection

Purpose:

This document provides a comprehensive functional specification for the Launch Pad Levels V2 ACSIL study. It serves as a complete reference for developers to understand, implement, or recreate the study's functionality.

Table of Contents

1. Executive Summary

Study Name: Launch Pad Levels - Multi-Timeframe Order Flow V2

Function Name: scsf_LaunchPadLevelsV2

Purpose:

The Launch Pad Levels study identifies price levels where aggressive order flow (market orders) exhibits directional imbalance while liquidity participation remains significant. These levels represent potential institutional accumulation or distribution zones that may act as launch pads for price movement.

Key Features:

- **Multi-Timeframe Analysis:** Configurable primary, confirmation, and structural timeframes with independent parameters
- **Weighted Blending:** Combines signals from multiple timeframes with user-defined weights
- **Order Flow Classification:** Automatically classifies trades as aggressive buy/sell using bid/ask, chart, and uptick rules
- **Time-Decay EWMA:** Exponentially weighted moving average with configurable half-life
- **Participation Weighting:** Normalizes intensity relative to baseline activity
- **Bucket-Based Aggregation:** Groups prices into configurable tick-size buckets
- **Real-Time Level Detection:** Identifies and ranks top levels by Launch Pad Score (LPS)
- **Visual Feedback:** Horizontal lines with optional labels and HUD display
- **Comprehensive Diagnostics:** Detailed logging of classification, timeframe metrics, and level statistics

2. Core Concept and Algorithm

2.1 High-Level Overview

The study operates on time & sales (T&S;) data, classifying each trade as aggressive buy or sell, then aggregating this order flow into price buckets. It tracks directional imbalance and intensity over multiple timeframes, scoring each level based on sustained imbalance, participation relative to baseline, and maturity (number of touches).

2.2 Processing Pipeline

Step	Description	Output
1. Data Ingestion	Receive T&S records on each bar update	Raw trade data
2. Classification	Determine aggressive buy vs sell using bid/ask, chart bid/ask, or split volume	Buy/Sell volume per bar
3. Bar Aggregation	Accumulate buy/sell volume for multiple timeframe windows	Per-TF buy/sell sums
4. Metrics Calculation	Calculate AI (Aggression Intensity) and DAS (Directional Aggression Score)	Per-TF AI, DAS
5. Baseline Normalization	Compute rolling mean AI for participation ratio	Per-TF: Participation weight
6. Price Bucketing	Map price to tick-based bucket, apply Winsorization	Bucket key (bin_ticks)
7. EWMA Update	Update bucket's EWMA_DAS and EWMA_AI with time decay	Updated bucket state
8. Level Scoring	Calculate LPS = |EWMA_DAS| × Participation × Maturity	LPS per bucket
9. Level Selection	Filter by threshold, distance, expiry; rank by LPS	Top-K qualified levels
10. Visualization	Draw horizontal lines with labels and HUD	Chart display

2.3 Key Algorithms

Exponentially Weighted Moving Average (EWMA):

```
EWMA_new = alpha × value + (1 - alpha) × EWMA_old
where alpha = 1 - exp(-ln(2) × time_delta / half_life)
```

Directional Aggression Score (DAS):

```
DAS = (buy_vol - sell_vol) / (buy_vol + sell_vol)
Range: [-1, +1] where +1 = all buy, -1 = all sell
```

Aggression Intensity (AI):

```
AI = buy_vol + sell_vol
Total aggressive volume (market orders) in the window
```

Launch Pad Score (LPS):

```
LPS = |EWMA_DAS| × (EWMA_AI / mean_AI) × maturity_factor
maturity_factor = min(touches / min_touches, 1.0)
```

3. Multi-Timeframe Architecture

3.1 Timeframe Definitions

The study supports three independent timeframes, each with configurable parameters:

Timeframe	Default Bars	Default Baseline	Default Weight	Default Min Touches	Default State
Primary	1	600s (10min)	1.0	10	Enabled
Confirmation	4	1200s (20min)	0.5	40	Enabled
Structural	20	3600s (60min)	0.3	200	Disabled

Bar Period: Automatically detected from chart (e.g., 15s for Renko 1 tick on fast instruments)

3.2 Weighted Blending Logic

Each enabled timeframe contributes to the blended metrics based on its weight:

```
weighted_contribution = Σ(TF_contribution × TF_weight) for enabled TFs
blended_ai = Σ(TF_ai × TF_weight) / Σ(TF_weight)
total_weight = Σ(TF_weight) for enabled TFs
```

3.3 Independent Baseline Calculation

Each timeframe maintains its own baseline window for participation ratio:

```
baseline_bars = baseline_seconds / seconds_per_bar
mean_AI = average(AI) over last baseline_bars
participation = current_AI / mean_AI (capped at 100x)
```

3.4 Contribution Calculation

Each timeframe's contribution combines directional signal and participation:

```
TF_contribution = DAS × min(participation, 100.0)
```

4. Data Structures

4.1 BarData Structure

Per-bar accumulator for order flow:

Field	Type	Description
buy_vol	float	Accumulated aggressive buy volume for this bar
sell_vol	float	Accumulated aggressive sell volume for this bar
valid	bool	True if bar has been touched by at least one trade

4.2 LevelBucket Structure

Price level tracking with EWMA state:

Field	Type	Description
bin_ticks	int	Bucket key (lower edge in ticks from arbitrary reference)
ewma_das	float	Exponentially weighted moving average of DAS
ewma_ai	float	Exponentially weighted moving average of AI
touches	int	Number of times this bucket has been updated
vol_sum	float	Total volume accumulated at this level
last_seen	SCDateTime	Timestamp of last update
active	bool	True if bucket is in use

4.3 PersistentData Structure

Main study state container (sc.GetPersistentPointer):

Field	Type	Description
schema_version	int	Version identifier (SCHEMA_VERSION = 4)
last_sequence	int64_t	Last processed T&S sequence number
last_trade_price	float	Previous trade price for uptick rule
current_bar_index	int	Circular buffer index for bars array
bars[MAX_BARS]	BarData[500]	Circular buffer of bar-level order flow
buckets[MAX_BUCKETS]	LevelBucket[2048]	Array of price level buckets
num_buckets	int	Number of active buckets
last_tick_size	float	Previous tick size (for change detection)

last_scale_override	float	Previous scale override value
total_trades	int	Classification diagnostic: total trades
classified_buy	int	Classification diagnostic: buy count
classified_sell	int	Classification diagnostic: sell count
unclassified	int	Classification diagnostic: unclassified count
method_bidask	int	Classification diagnostic: bid/ask method usage
method_chart	int	Classification diagnostic: chart bid/ask usage
method_uptick	int	Classification diagnostic: uptick rule usage

4.4 TimeframeConfig Structure (Local)

Per-timeframe configuration and state (stack-allocated in main function):

Field	Type	Description
enabled	bool	Whether this timeframe is active
bars	int	Number of bars to aggregate
baseline_seconds	int	Baseline window in seconds
baseline_bars	int	Baseline window in bars (calculated)
min_touches	int	Minimum touches for maturity
weight	float	Blending weight for this timeframe
buy_vol	float	Accumulated buy volume in window (calculated)
sell_vol	float	Accumulated sell volume in window (calculated)
ai	float	Aggression Intensity (calculated)
das	float	Directional Aggression Score (calculated)
mean_ai	float	Baseline mean AI (calculated)
participation	float	Participation ratio (calculated)
contribution	float	Weighted contribution (calculated)

5. Input Parameters

5.1 Primary Timeframe Parameters (Inputs 0-4)

Input	Name	Type	Default	Range	Description
0	Enable Primary Window	YesNo	Yes	-	Enable/disable primary timeframe
1	Primary: Bars	Int	1	1-50	Number of bars to aggregate
2	Primary: Weight	Float	1.0	0.0-10.0	Blending weight
3	Primary: Baseline (seconds)	Int	600	60-7200	Baseline window for participation
4	Primary: Min Touches	Int	10	1-500	Maturity threshold

5.2 Confirmation Timeframe Parameters (Inputs 5-9)

Input	Name	Type	Default	Range	Description
5	Enable Confirmation Window	YesNo	Yes	-	Enable/disable confirmation timeframe
6	Confirmation: Bars	Int	4	1-50	Number of bars to aggregate
7	Confirmation: Weight	Float	0.5	0.0-10.0	Blending weight
8	Confirmation: Baseline (seconds)	Int	1200	60-7200	Baseline window for participation
9	Confirmation: Min Touches	Int	40	1-500	Maturity threshold

5.3 Structural Timeframe Parameters (Inputs 10-14)

Input	Name	Type	Default	Range	Description
10	Enable Structural Window	YesNo	No	-	Enable/disable structural timeframe
11	Structural: Bars	Int	20	1-100	Number of bars to aggregate
12	Structural: Weight	Float	0.3	0.0-10.0	Blending weight
13	Structural: Baseline (seconds)	Int	3600	60-7200	Baseline window for participation
14	Structural: Min Touches	Int	200	1-1000	Maturity threshold

5.4 Global Study Parameters (Inputs 15-29)

Input	Name	Type	Default	Range	Description
15	Bucket Size (ticks)	Int	4	1-64	Price bucket width in ticks
16	Decay Half-Life (hours)	Float	8.0	0.5-48.0	EWMA half-life
17	LPS Threshold to Highlight	Float	0.40	0.0-2.0	Minimum LPS to display

18	Top-K Levels to Draw	Int	3	0-20	Maximum number of levels shown
19	Winsor Cap	Float	0.95	0.10-1.00	Percentile cap for price bucketing
20	Show Labels	YesNo	Yes	-	Show text labels on lines
21	Update Always	YesNo	Yes	-	Process on every tick
22	Fade Minutes	Int	60	1-1440	Age to fade line visually
23	Expire Minutes	Int	120	1-1440	Age to remove level
24	Max Dist Ticks	Int	200	1-10000	Maximum distance from current price
25	Always Top Pick	YesNo	Yes	-	Override distance for #1 level
26	Reset Now	YesNo	No	-	Clear all state (toggle to reset)
27	Tick Scale Override	Float	1.0	0.1-10.0	Manual tick size multiplier
28	Invert Classification	YesNo	No	-	Swap buy/sell for debugging
29	Show Detailed Stats	YesNo	No	-	Log detailed diagnostics

6. Order Flow Classification

6.1 Classification Hierarchy

The study uses a three-tier classification system to determine if each trade is an aggressive buy or sell:

Priority	Method	Condition	Classification
1 (Highest)	T&S Bid/Ask	Trade at ask	BUY
		Trade at bid	SELL
2	Chart Bid/Ask	Trade >= sc.Ask[Index]	BUY
		Trade <= sc.Bid[Index]	SELL
3 (Fallback)	Uptick Rule	Price > last_price	BUY
		Price < last_price	SELL
		Price == last_price	UNCLASSIFIED

6.2 T&S; Type Field Detection

The study automatically detects the T&S; configuration based on SC_TS_BIDTYPE field:

- **SC_ASKVOL_UINT:** Trade at Ask (BUY)
- **SC_BIDVOL_UINT:** Trade at Bid (SELL)
- **Empty/Invalid:** Falls back to chart bid/ask or uptick rule

6.3 Inversion Option

Input parameter 'Invert Classification' (Input 28) swaps buy/sell classification. Useful for debugging data feed issues or testing inverse strategies.

6.4 Diagnostic Tracking

The study maintains counters in PersistentData:

- **total_trades:** All processed trades
- **classified_buy:** Trades classified as aggressive buy
- **classified_sell:** Trades classified as aggressive sell
- **unclassified:** Trades that couldn't be classified
- **method_bidask:** Count using T&S; bid/ask method
- **method_chart:** Count using chart bid/ask method
- **method_uptick:** Count using uptick rule

7. Calculation Logic

7.1 Main Processing Flow

- **Initialization:** On first call or reset, initialize PersistentData structure
- **Tick Size Detection:** Get tick size from sc.TickSize, apply manual override if set
- **T&S; Processing:** Loop through all records in sc.TimeSalesArray since last_sequence
- **Classification:** For each trade, determine buy/sell/unclassified
- **Bar Accumulation:** Add volume to current bar's buy_vol or sell_vol
- **Timeframe Aggregation:** For each enabled TF, sum bars over window
- **Metrics Calculation:** Compute AI, DAS, mean_AI, participation for each TF
- **Blending:** Calculate weighted_contribution and blended_ai
- **Price Bucketing:** Determine bucket key from current price
- **EWMA Update:** Update bucket's ewma_das and ewma_ai with time decay
- **Level Scoring:** Calculate LPS for all active buckets
- **Filtering & Ranking:** Apply threshold, distance, expiry; sort by LPS
- **Visualization:** Draw horizontal lines and HUD text

7.2 Timeframe Window Aggregation

For each enabled timeframe, the study aggregates order flow over the specified number of bars:

```
Example: Primary window = 1 bar
1. Identify current_bar_index in circular buffer
2. Sum buy_vol and sell_vol from bars[current_bar_index] (just 1 bar)
3. Calculate AI = buy_vol + sell_vol
4. Calculate DAS = (buy_vol - sell_vol) / AI (handle AI=0 case)
```

```
Example: Confirmation window = 4 bars
1. Identify last 4 bars in circular buffer (wrapping around MAX_BARS)
2. Sum buy_vol and sell_vol from each of the 4 bars
3. Calculate AI and DAS from aggregated sums
```

7.3 Baseline Mean AI Calculation

For participation ratio, each timeframe calculates mean AI over its baseline window:

```
baseline_bars = baseline_seconds / seconds_per_bar
Sum AI over last baseline_bars (wrapping circular buffer)
mean_ai = sum / count (count = min(baseline_bars, valid bars))
If mean_ai < 1.0, set to 1.0 (avoid division by zero)
```

7.4 Participation Ratio

```
participation = current_ai / mean_ai
Capped at 100.0 to prevent extreme outliers from dominating LPS
```

7.5 Contribution Calculation

```
TF_contribution = DAS × min(participation, 100.0)
This combines directional signal (DAS) with intensity relative to baseline (participation)
```

7.6 Weighted Blending

```
weighted_contribution = Σ(TF_contribution × TF_weight) for enabled TFs
blended_ai = Σ(TF_ai × TF_weight) / total_weight
total_weight = Σ(TF_weight) for enabled TFs
```

```
If no TFs enabled, all values = 0
```

7.8 Price Bucketing with Winsorization

Converts continuous price to discrete bucket key:

```
1. effective_tick_size = sc.TickSize × tick_scale_override
2. ticks_from_zero = round(price / effective_tick_size)
3. If bucket_size_ticks > 1:
- Winsorize: modulo = ticks_from_zero % bucket_size_ticks
- If modulo > winsor_cap × bucket_size_ticks: round up to next bucket
- Else: floor to current bucket
4. bin_ticks = (ticks_from_zero / bucket_size_ticks) × bucket_size_ticks
```

```
Example: bucket_size=4, winsor_cap=0.95, price at tick 17
modulo = 17 % 4 = 1 (within first 5% of bucket)
Since 1 < 0.95×4=3.8, use floor: bin_ticks = 16
```

7.9 EWMA Update with Time Decay

For the bucket matching current price:

```
time_delta_seconds = (current_time - bucket.last_seen).GetTimeInSeconds()
alpha = 1.0 - exp(-0.693147 × time_delta_seconds / (half_life_hours × 3600))
bucket.ewma_das = alpha × weighted_contribution + (1-alpha) × bucket.ewma_das
bucket.ewma_ai = alpha × blended_ai + (1-alpha) × bucket.ewma_ai
bucket.touches += 1
bucket.vol_sum += blended_ai
bucket.last_seen = current_time
```

The alpha calculation ensures proper exponential decay based on actual time elapsed, making the study robust to replay speed and irregular bar timing.

8. Level Scoring (LPS)

8.1 LPS Formula

Launch Pad Score combines three factors:

```
LPS = imbalance × participation × maturity

Where:
• imbalance = |bucket.ewma_das| (absolute directional score)
• participation = bucket.ewma_ai / display_mean_ai (intensity ratio)
• maturity = min(bucket.touches / effective_min_touches, 1.0)
```

8.2 Effective Min Touches

The study uses the minimum of enabled timeframes' min_touches values:

```
effective_min_touches = min(TF.min_touches) for enabled TFs
If no TFs enabled, defaults to 1
```

This ensures levels aren't penalized for maturity when using multiple timeframes with different touch requirements.

8.3 Display Mean AI

For consistent participation display, uses the primary timeframe's mean AI if enabled, otherwise first enabled timeframe:

```
if (primary enabled): display_mean_ai = primary.mean_ai
else if (confirmation enabled): display_mean_ai = confirmation.mean_ai
else if (structural enabled): display_mean_ai = structural.mean_ai
else: display_mean_ai = 1.0
```

8.4 Directional Classification

Each level is classified as buy or sell based on EWMA_DAS sign:

```
if (bucket.ewma_das > 0): level is BUY (green)
if (bucket.ewma_das < 0): level is SELL (red)
```

9. Visual Output

9.1 Horizontal Lines

The study draws horizontal lines for each qualified level using sc.UseTool:

Property	Value	Description
DrawingType	DRAWING_HORIZONTALLINE	Infinite horizontal line
LineNumber	100000 + rank	Unique ID for each level (0-19)
BeginValue	level.price	Y-coordinate (price level)
Color	COLOR_GREEN or COLOR_RED	Buy=green, Sell=red
LineWidth	2 (solid) or 1 (faded)	Thicker for fresh levels
LineStyle	LINESTYLE_SOLID or LINESTYLE_DASH	Dashed when faded
AddMethod	UTAM_ADD_OR_ADJUST	Create or update existing line

9.2 Labels

When 'Show Labels' is enabled (Input 20), each line displays:

```
Format: 'LPS {lps:.2f} {BUY|SELL} hits {touches} age {age}m dist {dist}t'
Example: 'LPS 0.82 BUY hits 45 age 23m dist 18t'
Alignment: Right-justified, vertically centered on line
```

9.3 HUD Display

Heads-Up Display in upper-right corner shows real-time metrics:

```
Format: 'TF:{status} TopK={k} lvls={n} | AI={ai:.0f} mAI={mai:.0f}({w:.2f}x) | CLASS: B:{b}% S:{s}% U:{u}%
| Top: {price1}({lps1}) {price2}({lps2}) ...'

Components:
• TF: Active timeframes (P=Primary, C=Confirmation, S=Structural)
• TopK: Maximum levels to display
• lvls: Total qualified levels above threshold
• AI: Current blended aggression intensity
• mAI: Baseline mean AI for participation
• w: Total weight from enabled timeframes
• CLASS: Classification percentages (Buy/Sell/Unclassified)
• Top: Price and LPS of top 3 levels
```

9.4 Fading and Expiry

State	Age Condition	Visual Appearance	Behavior
Fresh	age < fade_minutes	Solid line, width 2	Normal display
Faded	fade_minutes ≤ age < expire_minutes	Dashed line, width 1	Still displayed but less prominent
Expired	age ≥ expire_minutes	Not drawn	Removed from display

10. Diagnostic Features

10.1 Detailed Stats Logging

When 'Show Detailed Stats' (Input 29) is enabled, the study logs comprehensive diagnostics every 300 ticks to the Sierra Chart Message Log:

Section	Content
Global Config	Bar period, schema version, bucket size, half-life, winsor cap, LPS threshold, TopK, expire/fade m
Timeframe Config	For each TF: enabled status, window size (bars/seconds), baseline (bars/seconds), min touches,
Timeframe Metrics	For each enabled TF: current AI, mean AI, participation ratio, DAS, contribution
Blended Output	Weighted contribution, total weight, blended AI
Classification	Total trades, buy/sell/unclassified percentages, method breakdown (bidask/chart/uptick)
Buckets	Total buckets, active non-expired buckets
Qualified Levels	Count of levels above threshold, top 10 levels with detailed metrics
Level Details	For each top level: rank, price, direction, LPS, EWMA_DAS, EWMA_AI, imbalance, participation,

10.2 Subgraph Outputs

The study provides two subgraphs for custom overlays or downstream studies:

Subgraph	Name	Value	Purpose
0	Probe	Current bar index % MAX_BARS	Circular buffer position (debugging)
1	BestDAS	Top level's EWMA_DAS	Strongest directional signal

10.3 Reset Functionality

Input 26 ('Reset Now') provides manual state clearing:

```
1. Toggle 'Reset Now' from No to Yes
2. Study calls p_Data->Init() to clear all buckets and classification counters
3. Sets last_sequence = -1 to reprocess T&S; data
4. User should toggle back to No after reset completes
```

10.4 Schema Versioning

The study includes automatic schema detection:

```
if (p_Data->schema_version != SCHEMA_VERSION):
- Call p_Data->Init() to clear incompatible state
- Prevents crashes from structure changes between versions
- Current SCHEMA_VERSION = 4
```

11. Success Criteria

11.1 Functional Requirements

- **Order Flow Processing:** Must correctly classify T&S; trades as buy/sell with >95% accuracy when bid/ask data available
- **Multi-Timeframe Support:** Must independently process primary, confirmation, and structural timeframes
- **Weighted Blending:** Must correctly blend timeframe signals according to user-defined weights
- **EWMA Calculation:** Must apply exponential decay based on actual time elapsed (replay-speed independent)
- **LPS Scoring:** Must calculate Launch Pad Score as |EWMA_DAS| × participation × maturity
- **Level Detection:** Must identify and rank levels by LPS, filtering by threshold, distance, and age
- **Visual Display:** Must draw horizontal lines at correct price levels with appropriate styling
- **HUD Updates:** Must display real-time metrics in heads-up display
- **Diagnostic Logging:** Must provide detailed statistics when enabled

11.2 Performance Requirements

- **Real-Time Processing:** Must process T&S; updates with <10ms latency on modern hardware
- **Memory Efficiency:** Total persistent state <500KB (BarData: 6KB + LevelBucket: 64KB + overhead)
- **Replay Speed:** Must handle 120X replay without accuracy degradation
- **CPU Usage:** Must not exceed 5% CPU during normal trading hours on 8-core system
- **Bucket Management:** Must efficiently handle up to 2048 price levels without performance degradation

11.3 Accuracy Requirements

- **Classification Accuracy:** >95% correct when bid/ask data present, >80% with uptick rule fallback
- **EWMA Precision:** Alpha calculation accurate to 6 decimal places
- **LPS Calculation:** All component factors (imbalance, participation, maturity) calculated correctly
- **Time Decay:** EWMA must properly decay regardless of replay speed or bar timing
- **Bucket Consistency:** Same price must always map to same bucket (deterministic bucketing)

11.4 Robustness Requirements

- **Tick Size Changes:** Must detect and adapt to tick size changes automatically
- **Division by Zero:** Must handle AI=0, mean_AI=0 cases gracefully
- **Data Gaps:** Must continue functioning after session breaks or disconnects
- **Invalid Data:** Must skip unclassified trades without crashing
- **Schema Changes:** Must auto-reset when schema version changes

12. Performance Considerations

12.1 Memory Layout

Structure	Size per Element	Count	Total Size	Notes
BarData	12 bytes	500	6 KB	Circular buffer for bar aggregation
LevelBucket	32 bytes	2048	64 KB	Price level tracking
PersistentData	-	1	~71 KB	Total study state
TimeframeConfig	56 bytes	3	168 bytes	Stack-allocated per call

12.2 Computational Complexity

Operation	Complexity	Frequency	Notes
T&S Classification	O(1) per trade	Per T&S record	Simple if/else logic
Bar Aggregation	O(W) per TF	Per bar	W = window size in bars (typically 1-20)
Baseline Calculation	O(B) per TF	Per bar	B = baseline size in bars (typically 40-240)
Bucket Search	O(N)	Per bar	N = num_buckets (typically <100 active)
EWMA Update	O(1)	Per bar	Single bucket update
Level Scoring	O(N)	Per display update	Score all buckets
Sorting	O(N log N)	Per display update	Sort by LPS (N typically <100)
Drawing	O(K)	Per display update	K = TopK (typically 3-10)

12.3 Optimization Strategies

- **Circular Buffer:** BarData uses modulo indexing to avoid shifting elements
- **Lazy Bucket Creation:** Only allocates buckets when price hits that level
- **Incremental Updates:** Only processes new T&S; records since last_sequence
- **Capped Participation:** Limits participation to 100x to prevent outlier dominance
- **Selective Logging:** Detailed stats only every 300 ticks when enabled
- **Fast Bucket Lookup:** Linear search sufficient for <100 active buckets
- **Pre-Computed Alpha:** EWMA alpha calculated once per bucket update

12.4 Scalability Limits

Limit	Value	Impact if Exceeded
MAX_BARS	500	Baseline window capped, older bars lost
MAX_BUCKETS	2048	Cannot track more price levels, study stops creating new buckets

Participation Cap	100x	Extreme spikes clipped to 100x baseline
Tick Scale Override	0.1-10.0	Outside range may cause incorrect bucketing
Timeframe Window	1-100 bars	Larger windows increase memory access time
Baseline Window	60-7200 seconds	Very large baselines increase computation time

13. Code Constants and Limits

13.1 Global Constants

Constant	Value	Type	Purpose
SCHEMA_VERSION	4	int	Data structure version for compatibility checks
MAX_BARS	500	int	Size of circular buffer for bar-level data
MAX_BUCKETS	2048	int	Maximum number of price level buckets
MAX_PARTICIPATION	100.0	float	Participation ratio cap to prevent outliers
MIN_MEAN_AI	1.0	float	Minimum baseline AI to avoid division by zero
LN_2	0.693147	float	Natural log of 2 for half-life calculation

13.2 Drawing Constants

Constant	Value	Purpose
LINE_NUMBER_BASE	100000	Base for horizontal line IDs (100000 + rank)
HUD_LINE_NUMBER	999997	Line ID for HUD text display
COLOR_GREEN	RGB(0,255,0)	Color for buy levels
COLOR_RED	RGB(255,0,0)	Color for sell levels
COLOR_WHITE	RGB(255,255,255)	Color for HUD text
LINE_WIDTH_FRESH	2	Line width for non-faded levels
LINE_WIDTH_FADED	1	Line width for faded levels

13.3 Input Parameter Ranges

Parameter	Min	Max	Default	Notes
Timeframe Bars	1	100	Varies by TF	Primary/Confirm: 1-50, Structural: 1-100
Timeframe Weight	0.0	10.0	Varies by TF	Primary: 1.0, Confirm: 0.5, Struct: 0.3
Baseline Seconds	60	7200	Varies by TF	1 min to 2 hours
Min Touches	1	1000	Varies by TF	Primary: 10, Confirm: 40, Struct: 200
Bucket Size Ticks	1	64	4	Price grouping width
Half-Life Hours	0.5	48.0	8.0	EWMA decay time
LPS Threshold	0.0	2.0	0.40	Minimum score to display
Top-K Levels	0	20	3	Maximum levels shown

Winsor Cap	0.10	1.00	0.95	Percentile for bucket rounding
Fade Minutes	1	1440	60	Age to fade visual
Expire Minutes	1	1440	120	Age to remove level
Max Dist Ticks	1	10000	200	Distance filter from current price
Tick Scale Override	0.1	10.0	1.0	Manual tick size multiplier

14. Usage Examples

14.1 Basic Setup (Default Configuration)

For initial testing, use default settings:

- Primary: 1 bar, weight 1.0, baseline 600s, min touches 10 - ENABLED
- Confirmation: 4 bars, weight 0.5, baseline 1200s, min touches 40 - ENABLED
- Structural: 20 bars, weight 0.3, baseline 3600s, min touches 200 - DISABLED
- Bucket Size: 4 ticks
- Half-Life: 8 hours
- LPS Threshold: 0.40
- TopK: 3 levels

This configuration provides fast primary signals with confirmation filtering, suitable for scalping or day trading on liquid instruments.

14.2 Aggressive Scalping Setup

For ultra-fast entries, emphasize primary timeframe:

- Primary: 1 bar, weight 2.0, baseline 300s, min touches 5 - ENABLED
- Confirmation: DISABLED
- Structural: DISABLED
- LPS Threshold: 0.30
- TopK: 5 levels
- Max Dist Ticks: 50

14.3 Conservative Swing Setup

For high-quality signals with strong confirmation:

- Primary: 4 bars, weight 1.0, baseline 1800s, min touches 20 - ENABLED
- Confirmation: 12 bars, weight 1.0, baseline 3600s, min touches 60 - ENABLED
- Structural: 40 bars, weight 0.5, baseline 7200s, min touches 300 - ENABLED
- LPS Threshold: 0.60
- TopK: 2 levels
- Fade Minutes: 120
- Expire Minutes: 240

14.4 Diagnostic / Debugging Setup

To investigate classification or calculation issues:

- Enable 'Show Detailed Stats' (Input 29)
- Check Message Log every 300 ticks for full diagnostics
- Monitor classification percentages in HUD
- If classification rates are poor, try 'Invert Classification' (Input 28)
- Use 'Reset Now' (Input 26) to clear state and restart tracking

15. Implementation Notes for Developers

15.1 Critical Implementation Details

• **Persistent State Management:** Use sc.GetPersistentPointer(1) with size check. Initialize on first call or schema version mismatch.

• **Circular Buffer Indexing:** current_bar_index = (current_bar_index + 1) % MAX_BARS. Always use modulo when accessing bars array.

• **T&S; Sequence Tracking:** Store last_sequence and only process records with sequence > last_sequence to avoid double-processing.

• **Time Delta Handling:** Always use SCDateTime.GetTimeInSeconds() for delta calculations. Never assume fixed bar periods.

• **EWMA Alpha Precision:** Use exp() function with proper natural log calculation: alpha = 1.0 - exp(-LN_2 * delta / half_life)

• **Division by Zero Protection:** Check AI > 0 before calculating DAS. Set mean_AI to max(computed_mean, 1.0).

• **Bucket Key Consistency:** Always use same effective_tick_size and bucket_size_ticks for entire session. Detect changes and reset if needed.

• **Drawing Tool IDs:** Use consistent line numbers (100000 + rank) to allow proper add/adjust operations.

• **Float Comparison:** Avoid exact float equality checks. Use fabsf(a - b) < epsilon where needed.

• **Input Validation:** SetIntLimits and SetFloatLimits enforce ranges. Don't rely on users staying within bounds in logic.

15.2 Common Pitfalls to Avoid

• **Forgetting Circular Buffer Wrap:** Always use modulo when calculating indices into bars array

• **Ignoring Schema Version:** Failing to check schema_version can cause crashes when structure changes

• **Hardcoded Bar Period:** Always calculate seconds_per_bar from sc.BaseDateTimeIn, don't assume 15s

• **Not Capping Participation:** Extreme ratios (>100x) will dominate LPS and cause false signals

• **Double-Processing T&S;:** Must track last_sequence to avoid re-processing same trades

• **Invalid Bucket Pointer:** Check bucket_idx < num_buckets before dereferencing buckets[bucket_idx]

• **Expired Level Display:** Always check age against expire_seconds before drawing

• **Zero Weight Division:** If all TFs disabled, total_weight = 0. Check before dividing for blended_ai.

15.3 Testing Recommendations

• **Replay Testing:** Test at 1X, 10X, 60X, and 120X replay speeds to verify time-independent decay

• **Data Feed Testing:** Test with T&S; bid/ask data, chart bid/ask only, and uptick rule fallback

• **Timeframe Combinations:** Test all 7 enable/disable combinations (P only, C only, S only, P+C, P+S, C+S, P+C+S)

• **Edge Cases:** Test with zero volume periods, tick size changes mid-session, and session gaps

• **Memory Leak Check:** Run for 8+ hours in replay to verify no memory growth

• **Classification Validation:** Compare classification results against known bid/ask data, expect >95% accuracy

• **LPS Calculation Spot Check:** Manually calculate LPS for a known bucket, verify study output matches

• **Visual Verification:** Confirm horizontal lines appear at correct price levels and fade/expire appropriately

16. Appendix

16.1 Formula Reference

EWMA Alpha:

```
alpha = 1 - exp(-ln(2) × Δt / t_half)
```

EWMA Update:

```
EWMA_new = α × value + (1-α) × EWMA_old
```

Aggression Intensity:

```
AI = buy_vol + sell_vol
```

Directional Aggression Score:

```
DAS = (buy_vol - sell_vol) / AI
```

Mean AI:

```
mean_AI = Σ(AI_i) / N over baseline window
```

Participation Ratio:

```
participation = min(AI / mean_AI, 100)
```

Timeframe Contribution:

```
contribution = DAS × participation
```

Weighted Blending:

```
blended = Σ(contribution_i × weight_i) / Σ(weight_i)
```

Maturity Factor:

```
maturity = min(touches / min_touches, 1.0)
```

Launch Pad Score:

```
LPS = |EWMA_DAS| × participation × maturity
```

Price to Ticks:

```
ticks = round(price / effective_tick_size)
```

Bucket Key:

```
bin_ticks = floor(ticks / bucket_size) × bucket_size
```

16.2 Glossary

ACSIL:

Advanced Custom Study Interface and Language - Sierra Chart's C++ API

AI (Aggression Intensity):

Total volume of aggressive orders (buy_vol + sell_vol)

DAS (Directional Aggression Score):

Signed imbalance ratio: (buy - sell) / (buy + sell), range [-1, +1]

EWMA:

Exponentially Weighted Moving Average - time-decayed average giving more weight to recent values

Half-Life:

Time for EWMA value to decay to 50% of initial value

Launch Pad Score (LPS):

Composite score = |DAS| × participation × maturity, identifies strong levels

Participation Ratio:

Current intensity relative to baseline: AI / mean_AI

Maturity Factor:

Confidence factor based on number of touches: min(touches / min_touches, 1.0)

Bucket:

Price level grouping that aggregates nearby prices into single zone

Timeframe:

Window of bars over which order flow is aggregated (primary, confirmation, structural)

T&S; (Time & Sales):

Tick-by-tick trade data with price, volume, timestamp, and bid/ask classification

Uptick Rule:

Fallback classification: buy if price > last_price, sell if price < last_price

Winsorization:

Rounding technique that clips outliers to bucket boundaries at specified percentile

HUD:

Heads-Up Display - on-chart text showing real-time metrics

Schema Version:

Data structure version number for detecting incompatible state after code changes

End of Functional Specification

This document provides a complete reference for implementing the Launch Pad Levels V2 ACSIL study. For questions or clarifications, refer to the source code comments or Sierra Chart ACSIL documentation.

Document generated: November 01, 2025 at 13:20:00